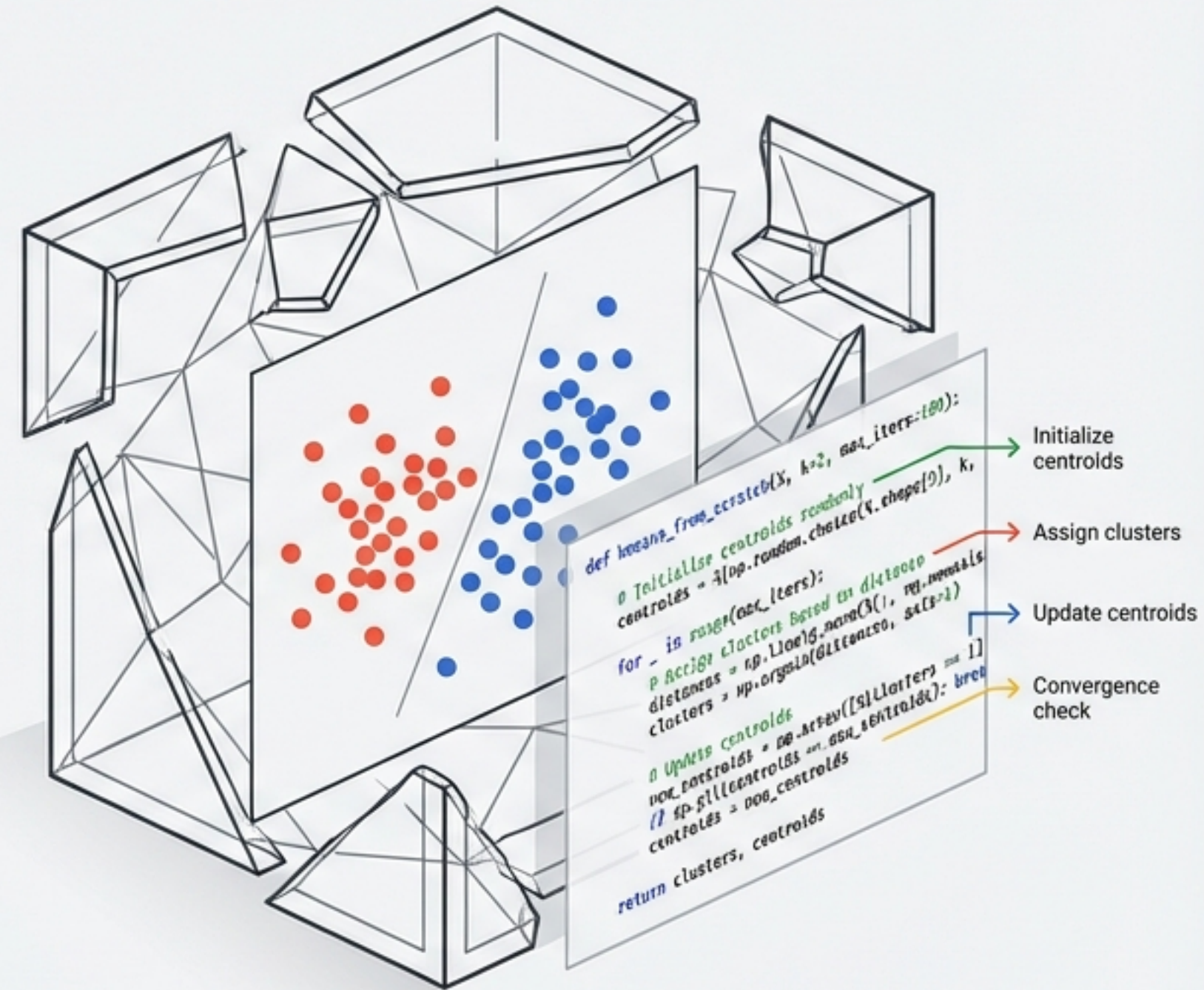


# Opening the Black Box

K-Means Clustering from Scratch in Python.





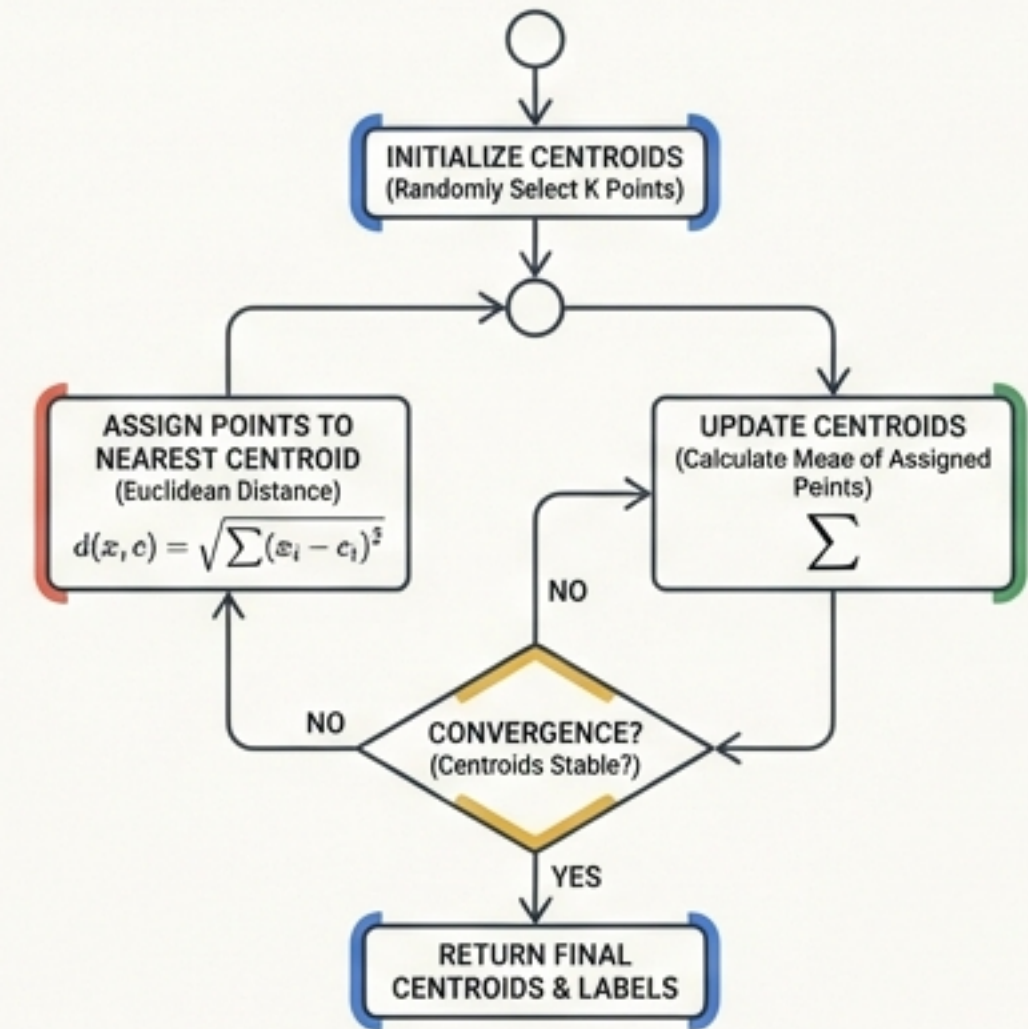
# True Understanding Means Building It Yourself

Importing an algorithm is easy. Coding it from scratch proves you actually understand the mathematics and logic. If you can build the engine, you own the machine.

```
model = KMeans(n_clusters=2)
```



**Usage**

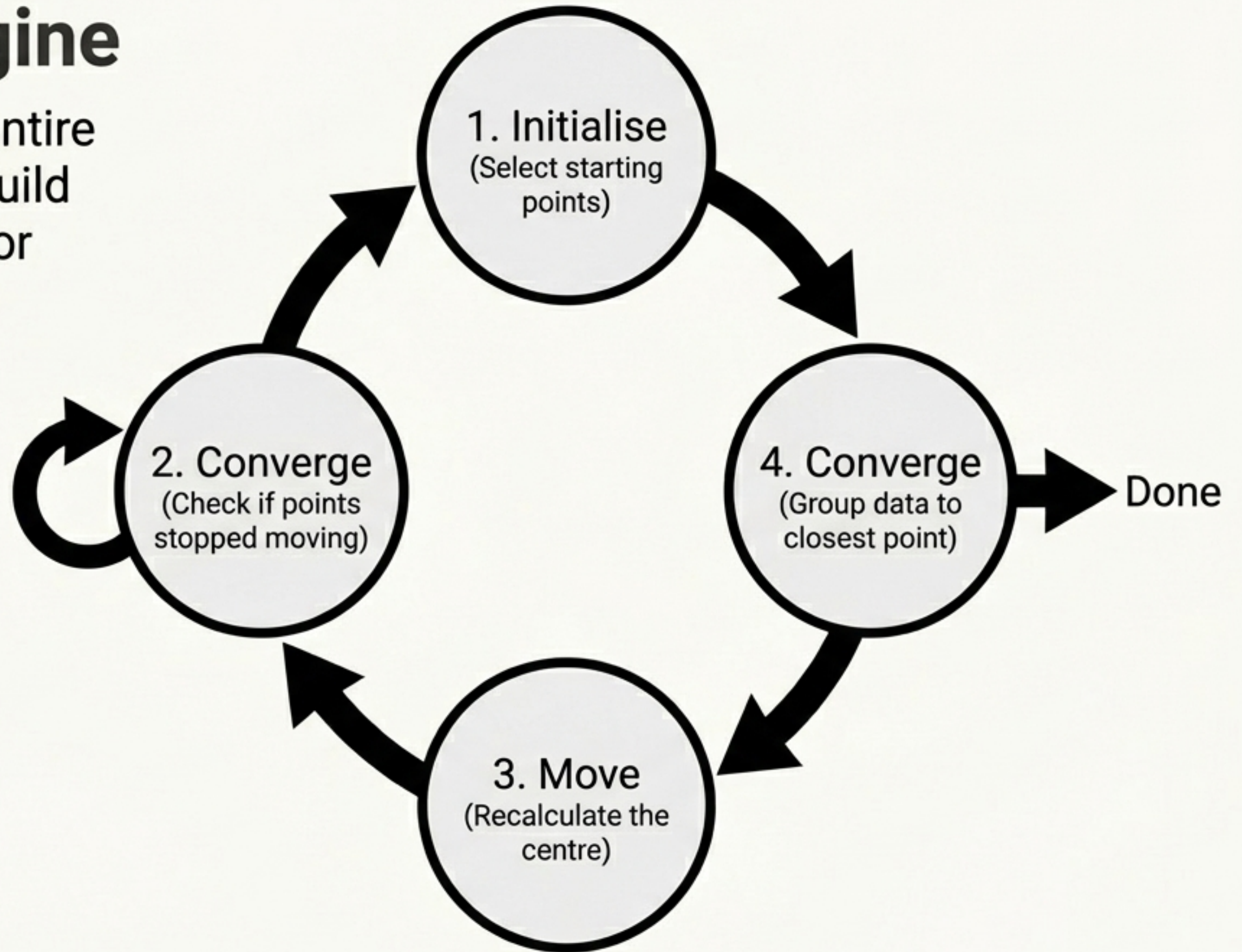


**Mastery**



# The K-Means Engine

Four logical gears drive the entire clustering process. We will build an Object-Oriented method for each.





# Setting the Foundation

We start by defining our class and default parameters. The algorithm needs to know how many clusters to hunt for, and how many times it is allowed to loop (max\_iter).

## **class KMeans**

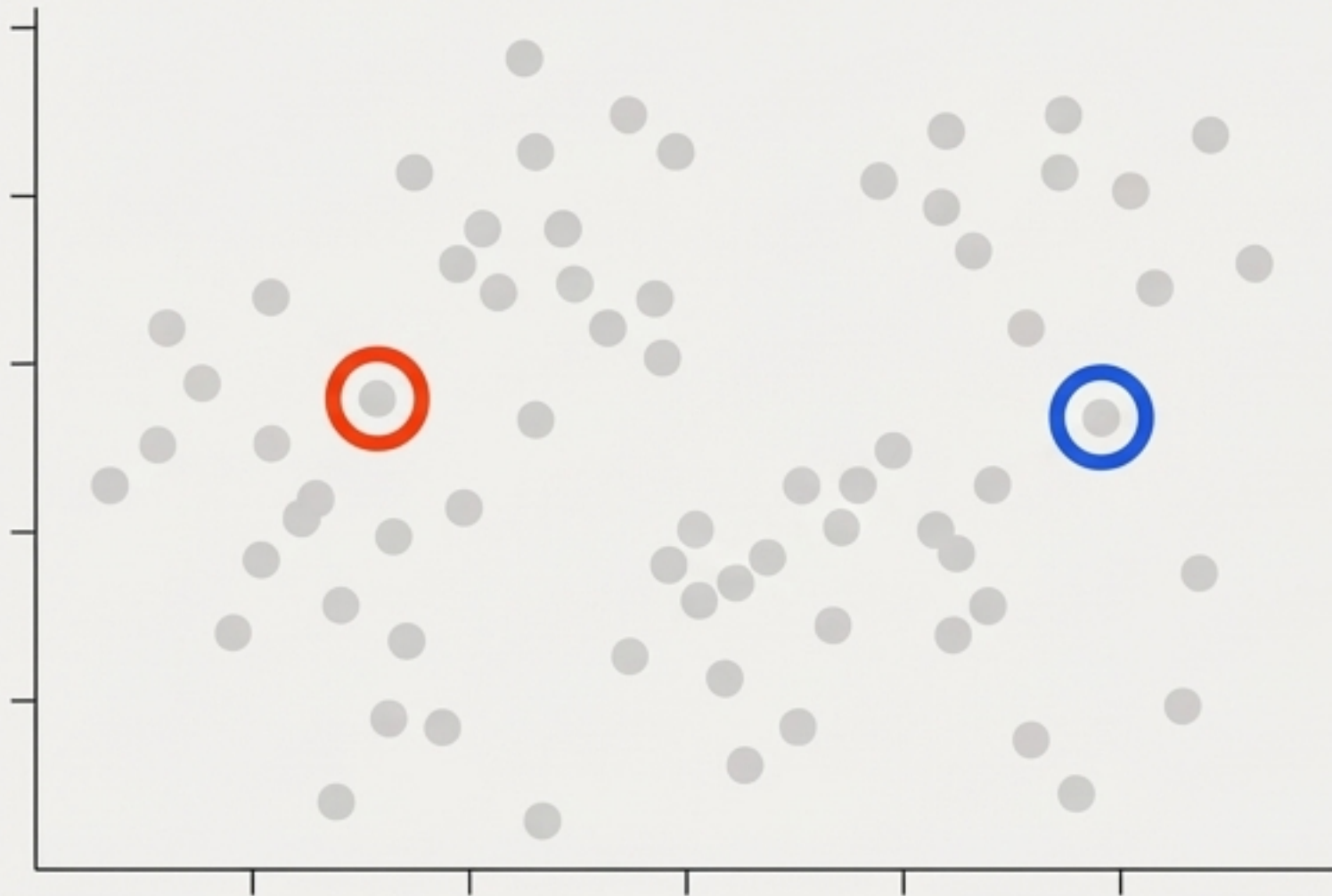
- n\_clusters
- max\_iter
- centroids

```
class KMeans:
    def __init__(self, n_clusters=2,
                  max_iter=100):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.centroids = None
```



# Step 1: Random Initialisation

We cannot guess the perfect centres immediately. Instead, we randomly sample `n_clusters` unique points directly from our dataset to act as our starting centroids.



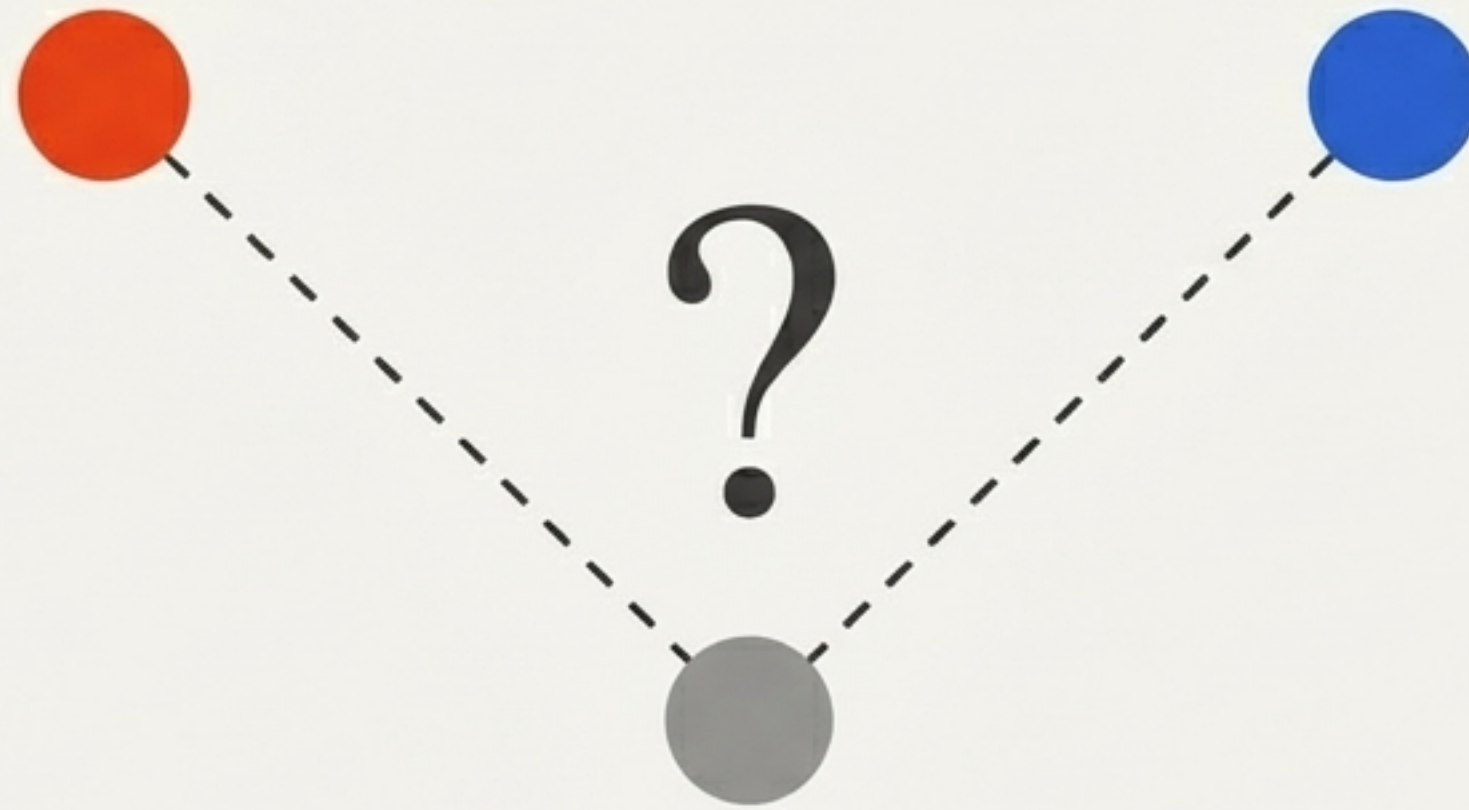
```
import random
```

```
random_index = random.sample(  
    range(X.shape[0]), self.n_clusters)  
self.centroids = X[random_index]
```



## Step 2: The Assignment Problem

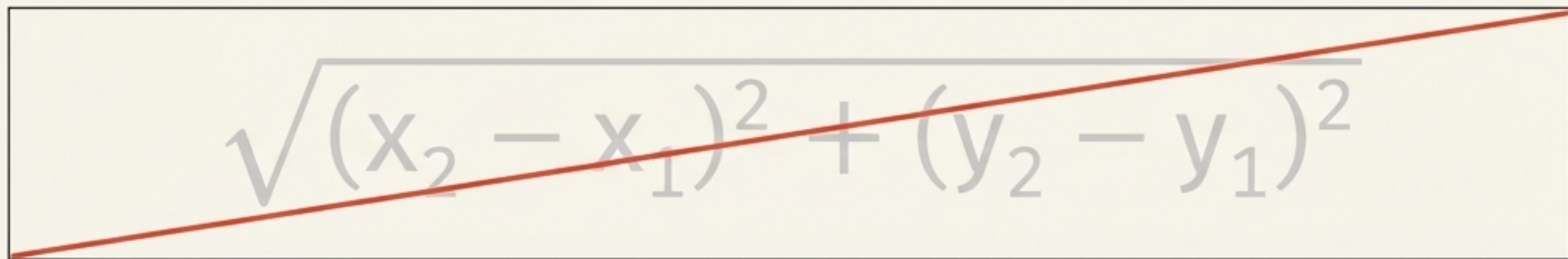
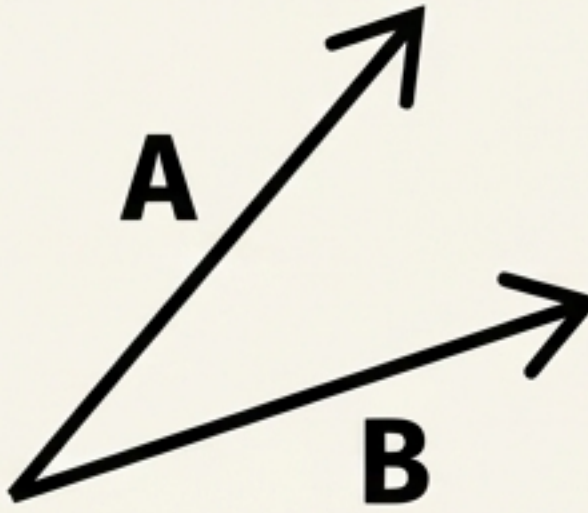
For every single row in our dataset, we must calculate its distance to every active centroid. The point is then assigned to the cluster of the nearest centroid.





# The N-Dimensional Math Trick

Writing out axis calculations manually fails as **feature dimensions grow**. By treating **coordinate differences as vectors** and using **the dot product**, our **algorithm dynamically handles infinite dimensions**.

|                                                                                                                                                         |                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|  $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$                              | <b>Breaks in 3D+</b>                                                                                            |
|  <pre data-bbox="816 1303 2515 1547">np.sqrt(np.dot(A-B, A-B))</pre> | <br>N-Dimensional<br>Scale |



# Executing the Assignment

We iterate through the active centroids, apply our n-dimensional dot product trick, and use `np.argmin` to capture the index of the closest cluster.



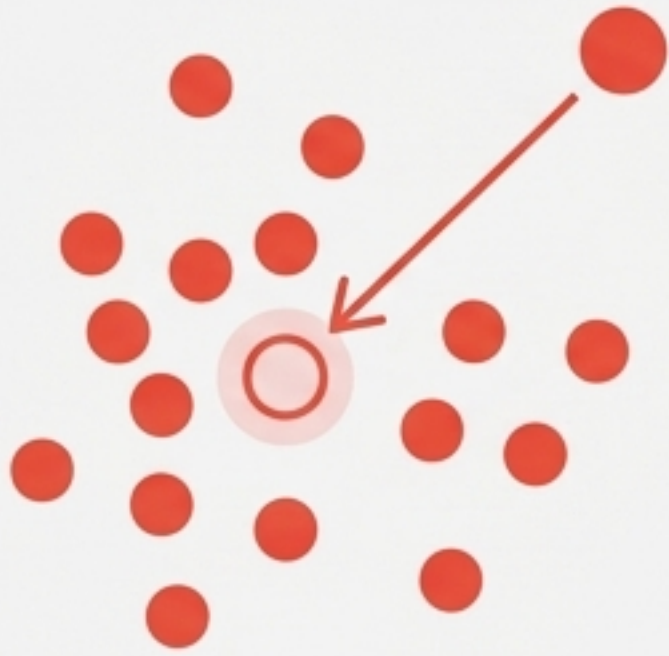
```
distances = []
for centroid in self.centroids:
    dist = np.sqrt(np.dot(row - centroid, row - centroid))
    distances.append(dist)

return np.argmin(distances)
```



# Step 3: Moving the Centroids

We isolate all data points assigned to a specific cluster ID. Calculating the mean across the columns (axis=0) gives us the exact coordinates for the new centroid position.

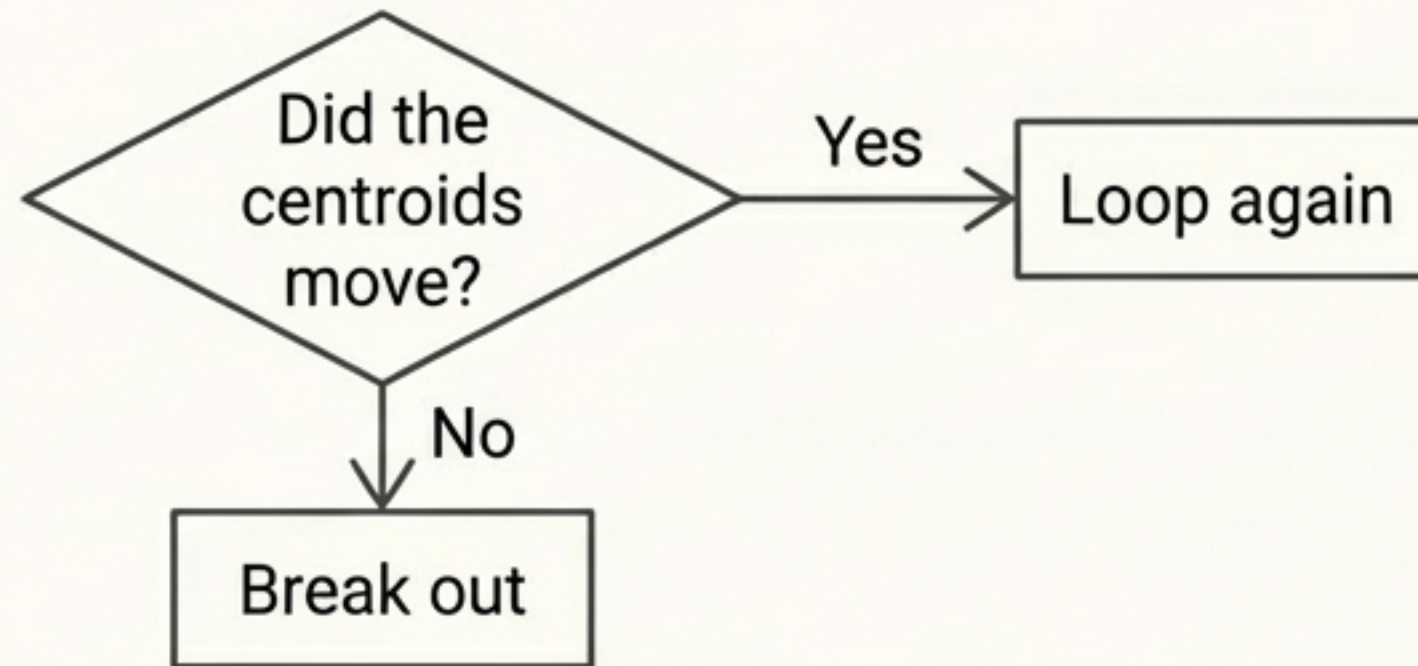


```
new_centroids = []  
for cluster_type in np.unique(cluster_group):  
    cluster_points = X[cluster_group == cluster_type]  
    new_centroids.append(np.mean(cluster_points, axis=0))
```



# Step 4: The Convergence Check

If the old\_centroids perfectly match the newly calculated centroids, it means the data points have stopped changing groups. The algorithm has converged.



```
if (old_centroids == self.centroids).all():  
    break
```



# The Loop in Action

The entire `fit()` method is simply our modular logic stacked inside a bounded loop.

Initialisation

```
def fit(self, X):  
    # Step 1  
    self.centroids = X[random.sample(...)] \
```

The Engine Loop

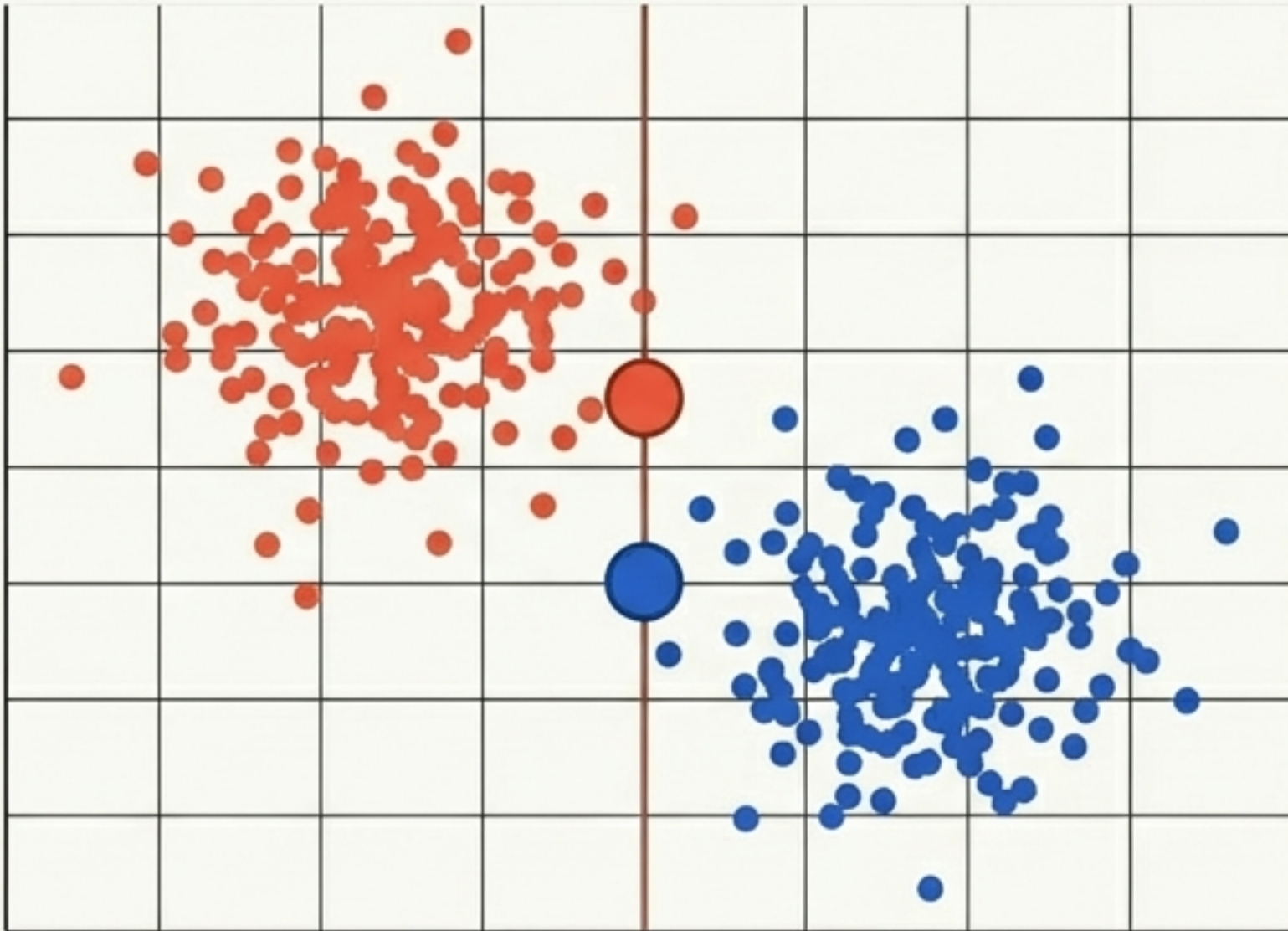
```
    for i in range(self.max_iter):  
        old_centroids = self.centroids  
        # Step 2  
        cluster_group = self.assign_clusters(X)  
        # Step 3  
        self.centroids = self.move_centroids(X, cluster_group)  
        # Step 4  
        if (old_centroids == self.centroids).all():  
            break
```



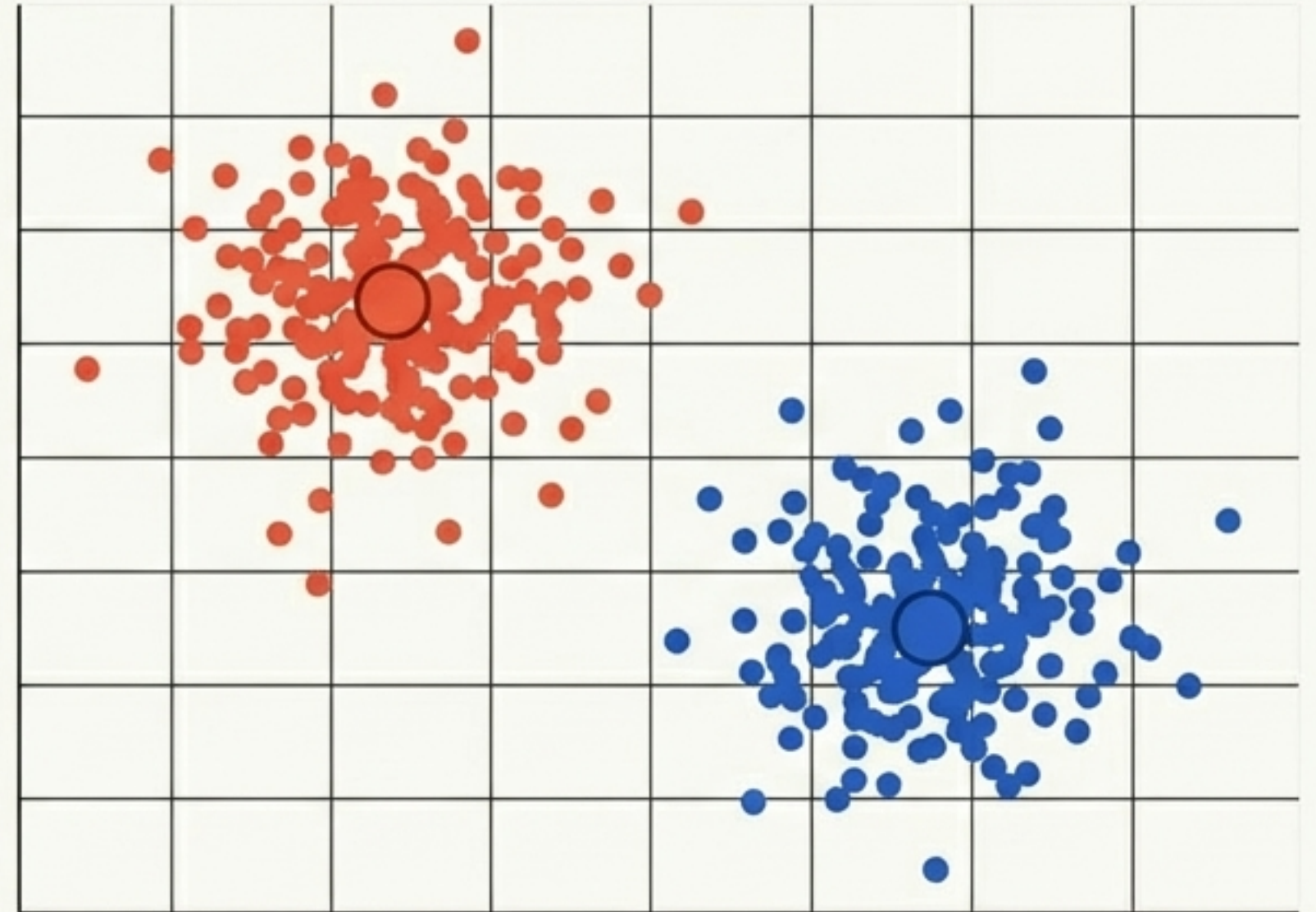
# The Impact of Iterations

Random initialisation means the first guess is rarely correct. A low iteration limit forces the algorithm to stop before the centroids find their true centre of mass.

`max_iter = 1`



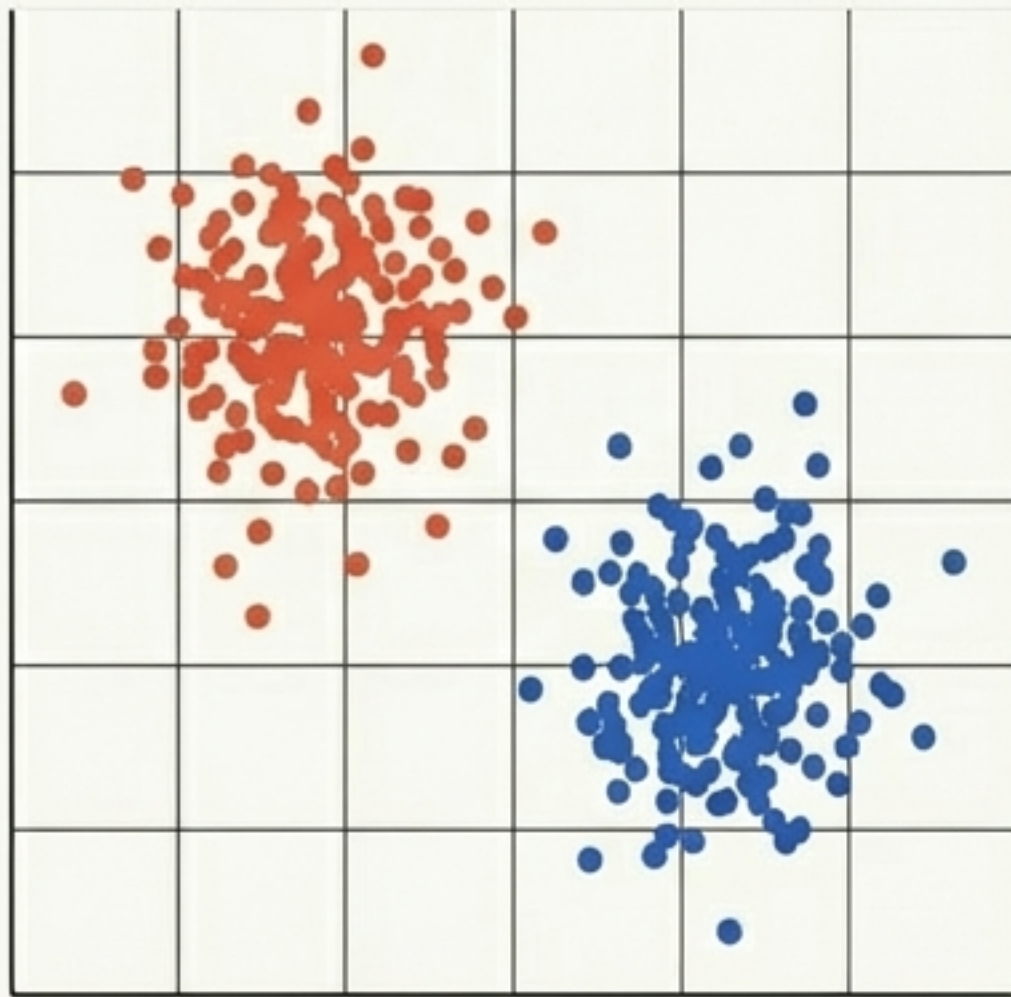
`max_iter = 100`



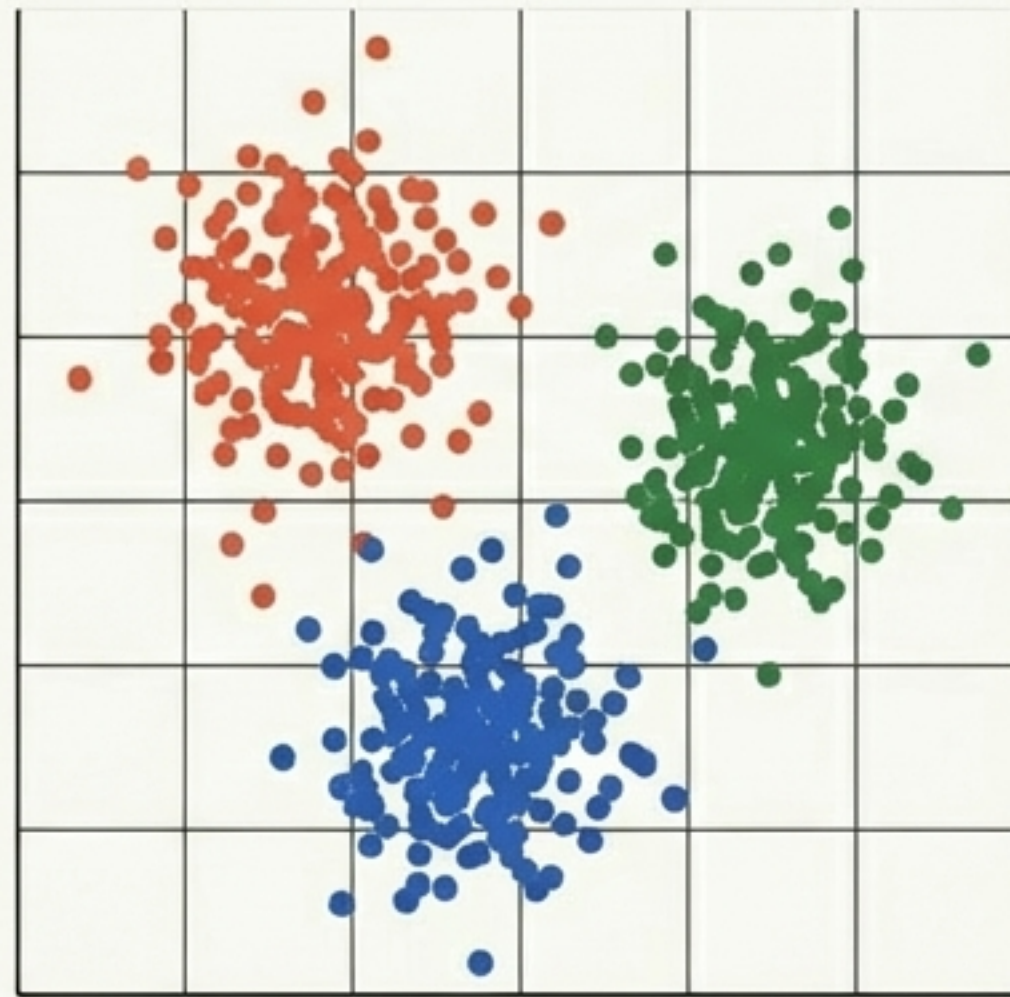


# Scaling Up Clusters

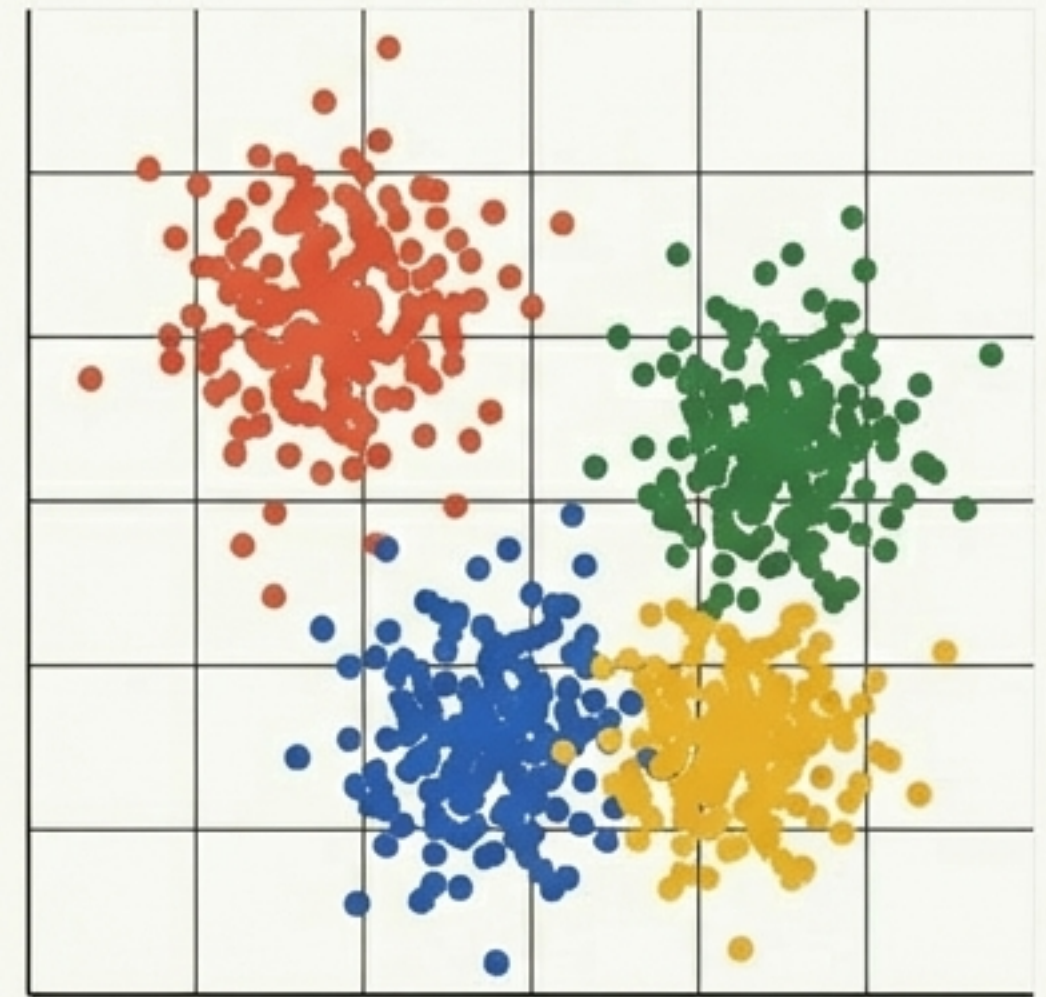
Because we avoided hardcoding dimensions or cluster counts, modifying `n_clusters` instantly adapts the random sampling and group assignments.



2 Clusters



3 Clusters

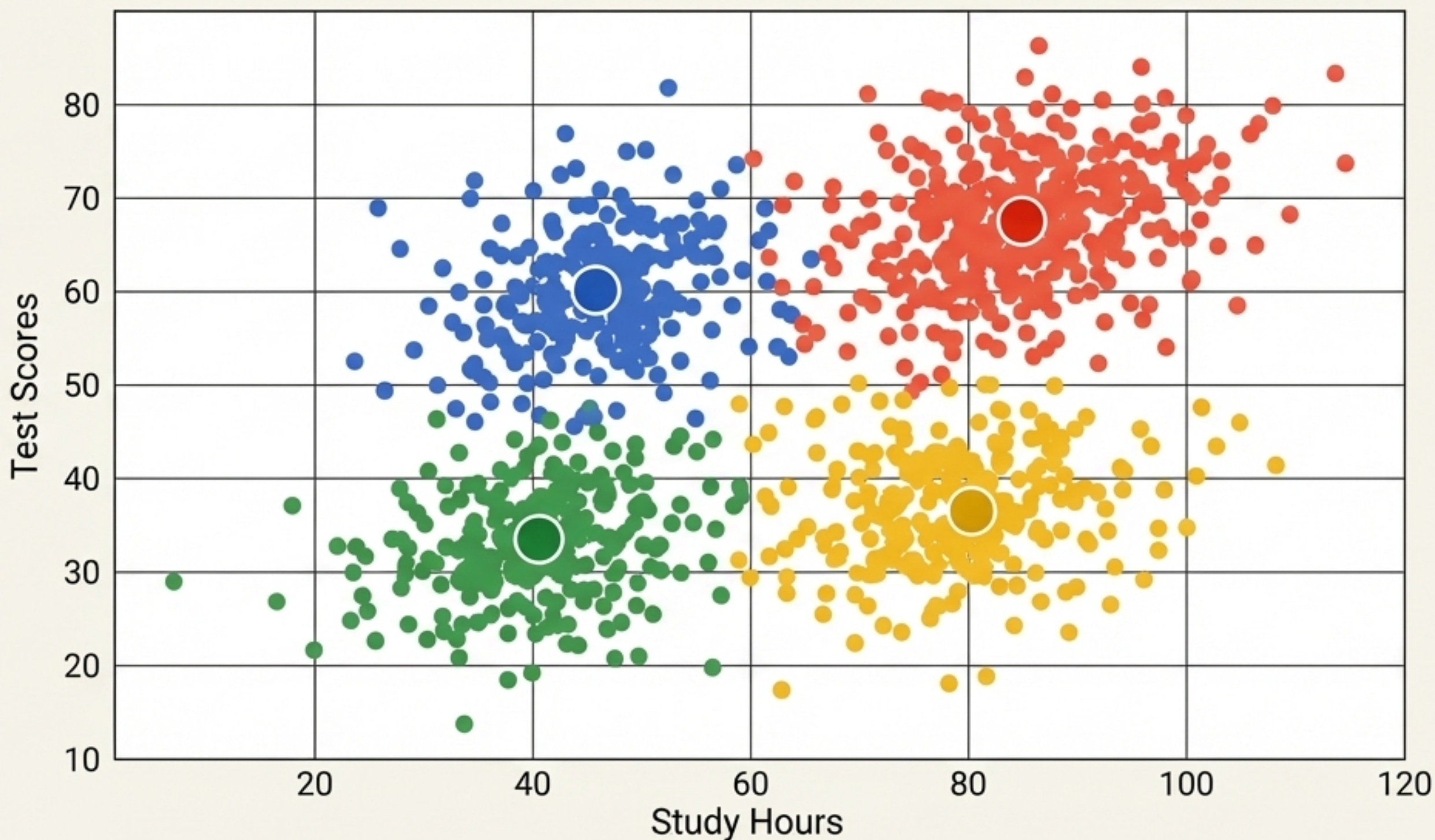


4 Clusters



# Beyond Toy Data: Real-World Application

The true test of a custom implementation is real data. Applied to the Student Interests dataset, our raw Python class successfully identifies hidden categorical boundaries.





# The Final Challenge

You now own the core engine. Your next step: implement the Within-Cluster Sum of Squares (WCSS) Elbow Method to automate the discovery of the perfect `n_clusters` parameter. Build it from scratch.

